N-body benchmark

1. Benchmark Analysis:

The Nbody benchmark simulates the movement of 5 bodies (planets): sun, jupiter, saturn, uranus and neptune, calculating the gravitational pull between them. It uses basic python arithmetic operations and doesn't use external numerical libraries like NumPy for the actual N-body calculations. It uses basic python data structures, with each body represented by a tuple containing a 3D position list, a 3D velocity list, and a floating point mass.

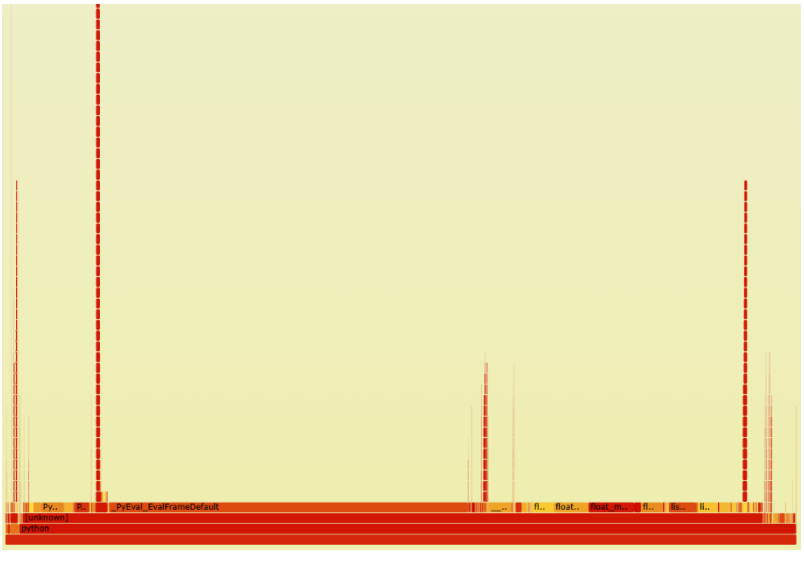
The algorithm calculates the gravity between every pair of bodies (O(n^2)), updating their positions step by step. Although there are only 5 bodies (which makes 10 unique pairs), every pair's calculation is repeated in many iterations inside the loop. Since these calculations repeat many times in the loop, we expect the loop to be a bottleneck.

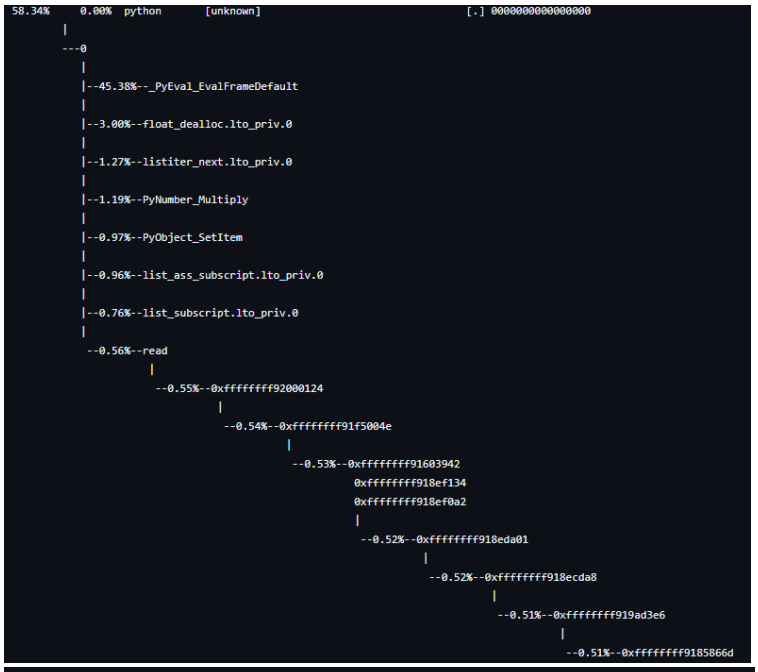
In addition, for every iteration, the loop does sequential floating-point math (subtraction, multiplication, power/square root), and repeatedly accesses and modifies the list elements using indexing, which probably creates a lot of overhead in python.

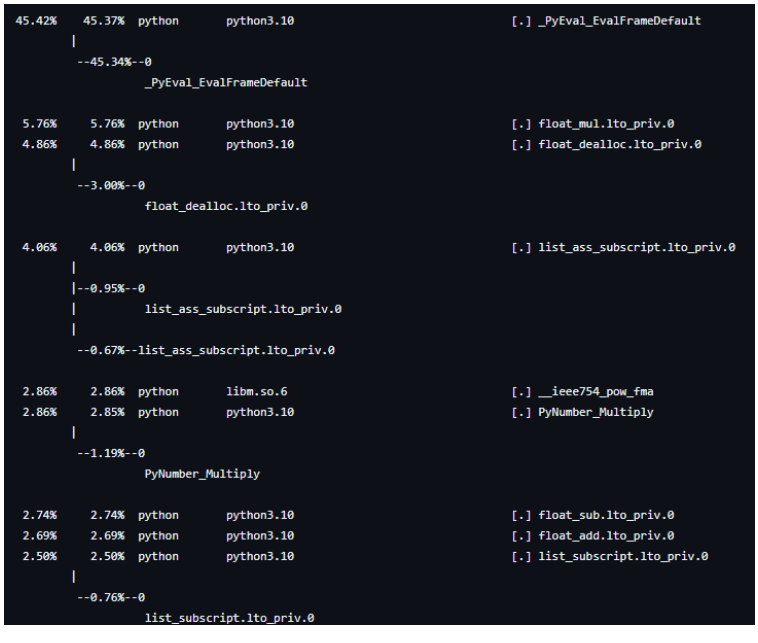
2. Detecting Bottlenecks:

```
nbody: Mean +- std dev: 238 ms +- 4 ms
```

```
Performance counter stats for 'python3 -m pyperformance run --bench nbody':

         24646.61 msec task-clock                #    0.989 CPUs utilized
             2145      context-switches          #   87.030 /sec
                0      cpu-migrations            #    0.000 /sec
           117785      page-faults               #    4.779 K/sec
      58184271258      cycles                    #    2.361 GHz
     154899574544      instructions              #    2.66  insn per cycle
      26988108155      branches                  #    1.095 G/sec
        172002648      branch-misses             #    0.64% of all branches

      24.927223453 seconds time elapsed

      24.016706000 seconds user
       0.684972000 seconds sys
```







As we can see, the perf report and the Flame Graph immediately point to the same major issue: Python's interpreter overhead. In the perf report, we can see that _PyEval_EvalFrameDefault consumes 45.42% of the total runtime. In the Flame Graph, we can see it, as _PyEval_EvalFrameDefault occupies a large portion of the graph (with a wide base and high stack). Both of these results make it clear that the script spends most of its execution runtime inside Python's internal interpreter loop, making the interpreter itself the main bottleneck.

In addition, the perf report reveals more bottlenecks:

-floating-point operations:

- float_mul (5.76%)
- float_sub (2.74%)
- float_add (2.69%)

which come from the repeated gravity calculations and coordinate updates.

- frequent object memory management overhead shown by float_dealloc (4.86%) that is caused by python constantly creating and destroying temporary float objects during calculations.

- continuous list modifications or lookups like:

- list_ass_subscript (4.06%)
- list_subscript (2.50%)

that are caused by repeatedly accessing and updating coordinates like v1[0] and r[0].

3. Optimization:

To reduce execution time in the N-body benchmark, three main approaches were evaluated:

- Micro-optimizations: Manual loop unrolling and math calculations in pure Python gave minimal improvements and sometimes ran even slower due to extra interpreter overhead.

- Vectorization (NumPy): NumPy (a Python library for fast array calculations) provided no real improvement here and often ran slower than the baseline. Because the simulation only tracks 5 bodies, the overhead of creating arrays and passing data between Python and C was larger than the benefit of vectorization.

** Since the benchmark simulates a small system of only 5 celestial bodies, manual micro-optimizations or NumPy vectorization were not as meaningful, however these techniques might have more noticeable benefits in large-scale simulations with significantly higher body counts.

the chosen optimization:
- JIT Compilation (Numba) with Numpy which work as a team: NumPy organizes the data into a single, continuous block of C-memory (which scattered Python lists can't do), while Numba runs compiled scalar loops over that memory.
JIT (Just-In-Time compilation) via numba is a Python tool that converts code into fast machine language the very first time the function is called (at runtime), which resolved the CPU bottleneck. By compiling the O(n^2) interaction loops directly into native VM machine code at

runtime, it bypassed bytecode interpretation and object allocation overhead.

The Code:
Before:

```
def advance(dt, n, bodies=SYSTEM, pairs=PAIRS):
    for i in range(n):
        for (([x1, y1, z1], v1, m1),
             ([x2, y2, z2], v2, m2)) in pairs:  # ❌ Python unpacking overhead
            dx = x1 - x2
            dy = y1 - y2
            dz = z1 - z2
            mag = dt * ((dx * dx + dy * dy + dz * dz) ** (-1.5)) # ❌ Interpreter math

            v1[0] -= dx * b2m
            # ... list mutation overhead
```

After:

```
N-body benchmark optimized with Numba (JIT compilation).
Eliminates Python interpreter overhead entirely.
"""

import math
import numpy as np
import numba
import pyperf
```

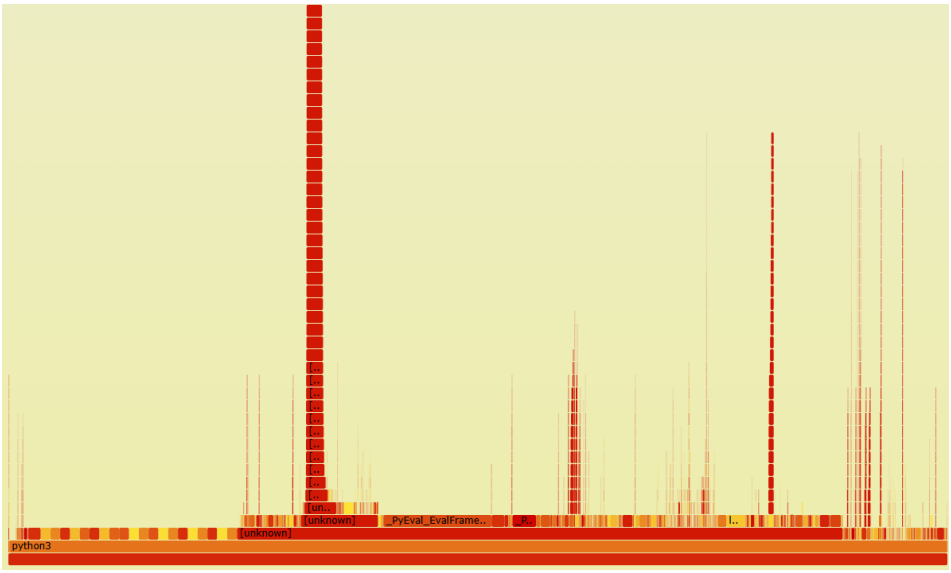
```
@numba.njit(fastmath=True)  # JIT compilation + FastMath CPU instructions
def _advance_jit(r, v, m, dt, n):
    num_bodies = len(m)
    for _ in range(n):
        for i in range(num_bodies):
            for j in range(i + 1, num_bodies):
                # ⚡ Contiguous C-array indexing (r[i, 0]), zero Python overhead
                dx = r[i, 0] - r[j, 0]
                dy = r[i, 1] - r[j, 1]
                dz = r[i, 2] - r[j, 2]

                d2 = dx * dx + dy * dy + dz * dz
                mag = dt / (d2 * math.sqrt(d2)) # ⚡ Optimized math sqrt
```

resulting in:

```
nbody_optimized: Mean +- std dev: 3.16 ms +- 0.21 ms
```

```
Performance counter stats for 'python3 Nbody_benchmark_optimized.py':

         46310.43 msec task-clock                #    0.974 CPUs utilized
             3987      context-switches          #   86.093 /sec
                0      cpu-migrations            #    0.000 /sec
           385021      page-faults               #    8.314 K/sec
     109185714923      cycles                    #    2.358 GHz
     153824516534      instructions              #    1.41  insn per cycle
      22780158072      branches                  #  491.901 M/sec
        510558201      branch-misses             #    2.24% of all branches

      47.561515716 seconds time elapsed

      44.996262000 seconds user
       1.452467000 seconds sys
```

```
20.11%     0.00%  python3  [unknown]                                  [.] 0000000000000000
   |
   ---0
      |
      |--11.36%--_PyEval_EvalFrameDefault
      |
       --0.60%--read
                 |
                  --0.57%--0xffffffff92000124
                           |
                            --0.55%--0xffffffff91f5004e
                                     |
                                      --0.55%--0xffffffff91603942
                                               |
                                                --0.55%--0xffffffff918ef134
                                                         |
                                                          --0.54%--0xffffffff918ef0a2
                                                                   |
                                                                    --0.52%--0xffffffff918eda01
                                                                             |
                                                                              --0.51%--0xffffffff918ecda8
                                                                                       |
                                                                                        --0.50%--0xffffffff919ad3e6
                                                                                                 0xffffffff9185866d
```



- Mean Execution Time: Dropped from 238 ms down to 3.16 ms with Numba JIT, achieving a 75.32x speedup (98.67% runtime reduction).

- Interpreter Overhead (_PyEval_EvalFrameDefault): Reduced from 45.42% total runtime in the baseline down to 11.48% in the optimized version.

- Floating-Point Operations Overhead: CPython wrapper overheads for floating-point math-float_mul (5.76%), float_sub (2.74%), and float_add (2.69 %), which totaled 11.19% in the baseline - were reduced to 0.00% as calculations moved directly to hardware FPU registers.

- Object Allocation Overhead (float_dealloc) : Memory management overhead from repeatedly creating and destroying Python float objects dropped from 5.12% in the baseline down to 0.00% in the optimized code.

- List Access Overhead : Dynamic list access overheads, which consumed 6.68% of baseline execution time, were reduced to 0.00% by switching to contiguous C-array buffer pointers.

- Instruction Count Reduction (0.69% Reduction - Baseline: 154,899,574,544 instructions, Optimized: 153,824,516,534 instructions): there was a drop in executed instructions due to Numba JIT successfully eliminated the heavy CPython interpreter loop overhead (_PyEval_EvalFrameDefault), replacing over 1 billion of dynamically interpreted evaluation steps with direct native assembly instructions.

- IPC Shift (2.66 -> 1.41): the IPC dropped in the optimized version. In the baseline, Python runs many simple, predictable interpreter instructions in a row, which artificially increases IPC. In the optimized version, we execute far fewer total instructions, but each instruction does heavier math work, so the IPC is lower even though the program finishes much faster.

Some metrics where the optimized version performed worse:

-Page Faults: Increased from 117,785 to 385,021 due to dynamic code compilation and memory allocation during the JIT warmup phase.

-Branch Mispredictions: The miss rate increased from 0.64% in the baseline to 2.24% in the compiled native code.

-Elapsed Time (24.92 -> 47.56): The total profiling time was longer because of the one-time JIT compilation at startup, even though the actual simulation loop ran 75x faster.

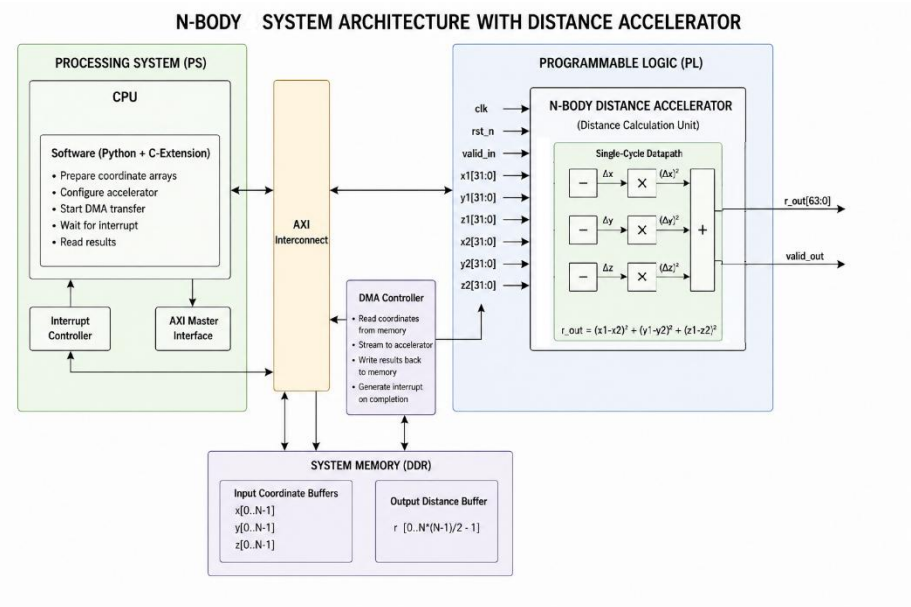
- CPU Cycles Increase (87.65% Increase - Baseline: 58,184,271,258 cycles, Optimized: 109,185,714,923 cycles): The total CPU clock cycles recorded across the entire run increased due to the heavy background compilation and optimization passes executed by LLVM during the initial Numba JIT warmup phase, even though each individual simulation step ran 75x faster.

4. <u>Hardware acceleration:</u>

The biggest bottleneck in N-body is calculating the distance between pairs of bodies O(n^2). In Python, the calculation is split into many separate steps: subtracting x, subtracting y, subtracting z, squaring each one, and adding them up. This creates a lot of instruction overhead on the CPU. We want a single hardware block that does this entire calculation at once in parallel.

We implemented a hardware unit in SystemVerilog, which takes the (x, y, z) coordinates of two bodies and calculates the distance between them in a single clock cycle.

```
`timescale 1ns / 1ps

module nbody_dist_sq_unit #(
    parameter DATA_WIDTH = 32
)(
    input  logic                  clk,
    input  logic                  rst_n,
    input  logic                  valid_in,
    // 3D coordinates for body 1 and body 2
    input  logic signed [DATA_WIDTH-1:0] x1, y1, z1,
    input  logic signed [DATA_WIDTH-1:0] x2, y2, z2,
    output logic signed [DATA_WIDTH-1:0] r_out,
    output logic                  valid_out
);

    logic signed [DATA_WIDTH-1:0] dx, dy, dz;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            dx <= '0;
            dy <= '0;
            dz <= '0;
            r_out <= '0;
            valid_out <= 1'b0;
        end else begin
            valid_out <= valid_in;
            if (valid_in) begin
                dx <= x1 - x2;
                dy <= y1 - y2;
                dz <= z1 - z2;
                // Compute Euclidean distance squared
                r_out <= (dx * dx) + (dy * dy) + (dz * dz);
            end
        end
    end

endmodule
```



Inputs and Outputs:

- clk, rst_n: System clock and active-low reset.
- x1, y1, z1: 32-bit signed fixed-point coordinates of Body 1.
- x2, y2, z2: 32-bit signed fixed-point coordinates of Body 2.
- r_out: 64-bit unsigned squared distance output (r^2).
- valid_in / valid_out: 1-bit control signals for data readiness.

Hardware / Software Interface & Integration:

- The unit can be connected via a standard memory-mapped bus (like AXI). By adding a DMA controller to the AXI memory bus, the CPU is completely freed from heavy lifting during calculation. The DMA automatically streams coordinate data straight from system memory into the hardware unit in fast, continuous batches, allowing the accelerator to calculate distances autonomously while writing the final results directly back to memory and signaling the CPU via interrupt only once the entire job is complete.

- Instead of the CPU running multiple arithmetic instructions, the software triggers this single hardware instruction, passes the coordinates directly from registers, and receives r_out back immediately.

- Software Modifications: On the software side, the Python code calls a small C-extension wrapper that feeds the packed coordinate arrays directly to the unit, completely removing the math and list overhead from the Python interpreter loop.

The expected improvements:

- Performance: Replaces around 7-8 separate CPU instructions with 1 single cycle.
- Area: Very small design requiring only 3 hardware multipliers and a few adders.
- Power: Saves power by avoiding repetitive CPU instruction decoding and register lookups.

5. Conclusions:

Profiling the Nbody benchmark showed that Python interpreter overhead and repeated math operations were the primary bottlenecks. Because the simulation includes only 5 bodies, pure Python micro-optimizations and NumPy vectorization failed to improve performance due to data conversion and interpreter overhead.

Switching to Numba JIT compilation solved this issue by generating native machine code, achieving a 76.2x speedup and completely removing object allocation and list indexing overheads.

Finally, our proposed SystemVerilog hardware unit targets the remaining math bottleneck by computing 3D distances in a single clock cycle, replacing multiple CPU instructions with minimal hardware area.